C++ 数组名的本质（结合指针，一眼吃透）

结合你刚学的**指针、地址、解引用**，我用**最核心的结论+通俗比喻+代码验证**，一次性讲透数组名的本质，这是指针和数组联动的关键！

一、一句话核心结论（必背）

数组名的本质：指向数组「首元素」的「常量指针」

拆解两个关键词：

1. **首元素**：数组名存储的是数组**第一个元素的内存地址**
2. **常量指针**：数组名的指向**不能修改**（不能让它指向别的变量/数组）

二、通俗比喻（延续酒店记忆法）

把数组想象成**一整排连续的酒店房间**：

- 数组 `int arr[5]` = 5个连续的房间，编号 `arr[0]`、`arr[1]...arr[4]`
- 数组名 `arr` = **第一个房间 `arr[0]` 的门牌号**
- 它就是一个**固定死、不能改的门牌号记录本**（常量指针）

三、代码实锤验证（最直观）

直接运行代码，证明数组名 = 首元素地址

Code block

```
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {10, 20, 30};  // 定义数组

    // 🔥 核心验证：数组名 arr 等价于 &arr[0]（首元素地址）
    cout << "数组名 arr 的值：" << arr << endl;
    cout << "首元素地址 &arr[0]：" << &arr[0] << endl;
    // 结果：两个地址 完 全 一 样！

```

```
    // 🔥 解引用数组名 = 首元素的值（和指针用法完全一致）
    cout << "解引用数组名 *arr: " << *arr << endl;  // 输出 10
    cout << "首元素 arr[0]: " << arr[0] << endl;    // 输出 10

    return 0;
}
```

四、关键特性：数组名是「常量指针」

数组名和普通指针**最大的区别**：

✅ 普通指针：可以随意修改指向

Code block

```
int a = 10;
int *p = arr;  // 指针指向数组
p = &a;        // ✅ 可以改指向，没问题
```

❌ 数组名：**不能修改指向**（因为是常量）

Code block

```
int a = 10;
arr = &a;      // ❌ 编译报错！数组名不能赋值、不能改指向
arr++;         // ❌ 编译报错！
```

五、数组名的指针运算（超好用）

既然数组名是指针，就可以做**指针加减运算**：

- `arr + 1` → 指向数组**第二个元素** `arr[1]`
- `arr + 2` → 指向数组**第三个元素** `arr[2]`
- 运算规则：**跳过 1 个元素的大小**（不是1字节）

Code block

```
// 验证指针运算
cout << *(arr + 1) << endl;  // 输出 20 等价于 arr[1]
cout << *(arr + 2) << endl;  // 输出 30 等价于 arr[2]
```

六、2个特殊情况（数组名≠首地址）

只有这两种情况，数组名**不是**首元素地址，其余场景全是指针：

1. `sizeof(arr)` → 计算**整个数组的总字节数**，不是指针大小
2. `&arr` → 取**整个数组的地址**（地址值和首元素一样，但含义不同）

Code block

```
cout << sizeof(arr) << endl;   // 3个int，输出 12（不是4）
```

七、终极总结（新手必背）

1. **数组名本质**：指向数组**首元素**的**常量指针**
2. **等价关系**：`arr == &arr[0]`，`*arr == arr[0]`
3. **核心限制**：数组名不能修改指向（不能赋值、不能`++/--`）
4. **通用用法**：数组名可以完全当作指针使用（解引用、指针运算）

一句话串起指针+数组

数组是连续的内存，数组名就是锁死在第一个元素上的指针，这就是C++数组的底层真相！

（注：文档部分内容可能由 AI 生成）